COMPARATIVE SENTIMENT ANALYSIS ON TWITTER USING CLASSICAL, DEEP LEARNING, AND TRANSFORMER-BASED MODELS

Supporting Material

Name: ARPAN NEUPANE

Student ID: 20015691

Email: Neupane-a1@ulster.ac.uk

MSc. Computer Science and Technology
Ulster University, Birmingham

Supervisor: Michael Ajao-Olarinoye

Table of Contents

1.0 Introduction and Synopsis

The overall objective of this project is to present supplementary evidence from the experimental phase of the study, including classification reports, confusion matrices, training logs, by implementing python scripts that can do:

"Comparative Sentiment Analysis on Twitter Using Classical, Deep Learning, and Transformer-Based Models, *within a set dataset"*

The research paper aims to ensure transparency and reproducibility of the research by offering direct access to outputs generated during the model training and evaluation process. It also serves to illustrate the differences in performance across classical machine learning, sequence-based deep learning, and transformer-based models when applied to a large-scale multiclass sentiment dataset.

The evidence contained here supports the claims made in the main paper regarding the superiority of fine-tuned transformer models (BERT, RoBERTa) compared to both frozen transformer variants and non-transformer baselines. Additionally, this paper highlights recurring issues, such as class imbalance and misclassification patterns, which provide insight into persistent challenges in sentiment classification tasks.

2.0. Project Aims and Objectives

The comprehensive aim of this project was to conduct a systematic evaluation of sentiment classification models, with a particular focus on comparing traditional machine learning, recurrent neural networks, and transformer-based architectures. This study aimed to answer which of the two approaches is more effective when it comes to the extraction of sentiment polarity in textual content on social media, and particular emphasis was placed on the accuracy and the degree of reliability in the case of different sentiment classes.

To achieve this, the project set out the following objectives:

2.1 Core Aims – "Must Haves"

- To build baseline sentiment classifiers using traditional machine learning methods (Naïve Bayes, Random Forest).
- To implement recurrent neural network models (LSTM and BiLSTM) and evaluate their effectiveness in handling sequential text data.
- To fine-tune transformer-based models (BERT and RoBERTa) for sentiment classification and compare their performance with non-transformer models.
- To conduct ablation studies (frozen vs. fine-tuned) to understand the contribution of contextual embeddings in classification tasks.
- To evaluate models using standard metrics including accuracy, precision, recall, F1-score, and confusion matrices.
- To identify patterns of misclassification, particularly with respect to class imbalance between negative, neutral, and positive sentiments.

2.2 Desirable Aims – "Could Haves"

- To compare computational efficiency (training time and resource usage) across classical, deep learning, and transformer models.
- To explore the interpretability of errors through confusion matrices and classification reports.
- To assess whether smaller models (BiLSTM, Random Forest) can provide competitive alternatives when resources are constrained.
- To investigate potential strategies to mitigate class imbalance, such as weighting schemes or data augmentation, for future work.
- To generate a clear comparative framework that could be used by other researchers for benchmarking sentiment analysis tasks.

3.0 Background Information and Literature Review

3.1 Review of Journal
[Esuli, A., & Sebastiani, F. (2006). SentiWordNet: A publicly available lexical resource for opinion mining]

This study [1] introduced SentiWordNet, a lexical database where each WordNet synset is annotated with sentiment scores (positive, negative, objective). It marked an important step in lexicon-based sentiment analysis, allowing polarity detection without supervised training. However, the method struggled with contextual nuances, sarcasm, and negations. This research provided the foundation for early sentiment systems but also highlighted the limitations that later motivated machine learning and neural approaches.

3.2 Review of Journal
[Pang, B., Lee, L., & Vaithyanathan, S. (2002). Thumbs up? Sentiment classification using machine learning techniques]

This paper [2] pioneered the application of supervised machine learning to sentiment classification, comparing Naïve Bayes, Maximum Entropy, and Support Vector Machines. It demonstrated that statistical models trained on bag-of-words features outperformed simple lexicon methods. However, the approach still lacked the ability to capture semantic relationships or long-range dependencies. Its significance lies in establishing machine learning as a core method for sentiment analysis, a baseline against which modern models are evaluated.

3.3 Review of Journal
[McCallum, A., & Nigam, K. (1998). A comparison of event models for Naïve Bayes text classification]

This foundational study [3] analysed Naïve Bayes event models, highlighting differences between multinomial and Bernoulli variants. It showed that despite strong independence assumptions, Naïve Bayes could achieve competitive accuracy in text classification tasks. Its relevance to this project lies in validating Naïve Bayes as a computationally efficient baseline, still useful in low-resource settings. However, its inability to model feature dependencies limits performance compared to more advanced methods.

3.4 Review of Journal
[Breiman, L. (2001). Random forests]

Breiman's work [4] introduced Random Forests, an ensemble method combining decision trees through bagging and feature randomness. The algorithm reduces overfitting and provides robust performance across high-dimensional datasets, including text. For sentiment analysis, Random Forests offered a balance between accuracy and interpretability. This project employs Random Forest as a classical benchmark, demonstrating that even non-neural methods remain competitive when tuned effectively.

3.5 Review of Journal
[Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory]

This landmark paper [5] presented LSTM networks, addressing the vanishing gradient problem in recurrent neural networks. LSTMs introduced gating mechanisms to retain information across longer sequences, making them suitable for tasks such as sentiment analysis where context spans multiple words. In this project, LSTM served as an initial deep learning baseline, though results showed limitations compared to BiLSTM and transformers.

3.6 Review of Journal

[Graves, A., & Schmidhuber, J. (2005). Framewise phoneme classification with bidirectional LSTM networks]

The authors [6] proposed Bidirectional LSTMs (BiLSTMs), which process text sequences in both directions. This architecture provides richer context and improves classification accuracy, particularly for tasks where sentiment depends on both prior and subsequent words. In this project, BiLSTM achieved strong results (84% accuracy), proving its effectiveness as a bridge between classical RNNs and transformers.

3.7 Review of Journal

[Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding]

BERT [7] revolutionised NLP with transformer-based contextual embeddings. Unlike RNNs, BERT captures dependencies across an entire sequence simultaneously using self-attention. Pre-trained on massive corpora, it requires fine-tuning for specific tasks. In this project, fine-tuned BERT achieved 85.8% accuracy, while frozen BERT collapsed, reinforcing that task-specific tuning is crucial for success.

3.8 Review of Journal

[Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., et al. (2019). RoBERTa: A robustly optimized BERT pretraining approach]

RoBERTa [8] refined BERT by training with larger data, dynamic masking, and removal of next-sentence prediction. These optimisations yielded state-of-the-art results across benchmarks. In this project, fine-tuned RoBERTa delivered the best accuracy (87.8%), outperforming all other models with balanced performance across sentiment classes. Its success highlights the value of large-scale pretraining and optimisation.

3.9 Literature Review Summary

No.	Author(s)	Type	Key Contribution
1	Esuli & Sebastiani (2006)	Journal	Introduced *SentiWordNet*; advanced lexicon-based methods but limited in handling context/negation.
2	Pang et al. (2002)	Journal	Applied supervised ML (NB, SVM) to sentiment; established ML as baseline.
3	McCallum & Nigam (1998)	Journal	Analysed Naïve Bayes event models; efficient baseline for text classification.
4	Breiman (2001)	Journal	Proposed Random Forests; ensemble learning robust for high-dimensional data.
5	Hochreiter & Schmidhuber (1997)	Journal	Introduced LSTM; solved vanishing gradient, enabling long-sequence learning.
6	Graves & Schmidhuber (2005)	Journal	Developed BiLSTM; bidirectional context improves sentiment classification.
7	Devlin et al. (2019)	Journal	Introduced BERT; transformer embeddings set new standard for NLP.
8	Liu et al. (2019)	Journal	Proposed RoBERTa; optimised pretraining, achieving best results in this project.

4.0 Life Cycle and Development Process

4.1 Data Collection

The dataset used in this project was obtained from a Kaggle-hosted Sentiment Analysis Dataset consisting of 48,133 annotated comments/ tweets labelled as positive, neutral, or negative. The dataset reflected typical characteristics of Twitter text, including abbreviations, slang, hashtags, etc. Proportional sampling was applied to maintain sentiment distribution across training and test sets. An 80:20 split was performed, with training data further divided into training and validation subsets.

4.2 Data Processing and Cleaning

The preprocessing pipeline implemented across Python scripts included:

- Text normalisation: removal of special characters, numbers, links, and extra spaces.
- Stop word removal: using the NLTK library.
- Tokenisation: word-level for LSTM/BiLSTM models, and subword-level (WordPiece/Byte-Pair Encoding) for transformers (BERT and RoBERTa).
- Padding/truncation: applied to ensure consistent sequence lengths (128 tokens for BERT/RoBERTa, ~100 for RNN-based models).
- Encoding: label encoding for sentiment categories (0 = Negative, 1 = Neutral, 2 = Positive).

For classical models, TF-IDF vectorisation was applied as implemented in "New_Classical models.py". For deep learning models, Keras tokenizers generated word indices. For transformers, HuggingFace's BertTokenizer and RobertaTokenizer were employed.

4.3 Creating Models Using Different Machine Learning Techniques

4.3.1 Classical Models (New_Classical models.py)

- Implemented Naïve Bayes and Random Forest using scikit-learn.
- Input features: TF-IDF vectors.
- Random Forest used 200 estimators with tuned max depth.
- Served as interpretable baselines, with Random Forest achieving ~84% accuracy, significantly outperforming Naïve Bayes (~68%).

4.3.2 Deep Learning Models (LSTM Sentiment.py, BiLSTM.py)

- LSTM: Embedding layer → LSTM (128 units) → Dense softmax layer.
    - Training configured with Adam optimizer, categorical crossentropy loss, batch size = 32.
    - Struggled with class imbalance, often predicting positives. Accuracy ~43%.
- BiLSTM: Embedding layer → Bidirectional LSTM (128 units, dropout 0.3) → Dense softmax.
    - Provided stronger context modelling by processing input in both directions.
    - Accuracy ~84%, demonstrating competitive performance with classical models.

4.3.3 Transformer Models (bert with ablation.py, RoBERTa with ablation.py)

- BERT (bert with ablation.py):
    - Backbone: bert-base-uncased.
    - Inputs prepared via BertTokenizer.
    - Architecture: Pre-trained BERT → Dropout → Dense classification layer.
    - Ablation: tested frozen vs fine-tuned variants. Frozen collapsed (~48%), fine-tuned achieved 85.8%.
- RoBERTa (RoBERTa with ablation.py):
    - Backbone: roberta-base.
    - Similar fine-tuning strategy with AdamW optimizer and learning rate warmup.
    - Ablation confirmed frozen RoBERTa weaker (~58%), while fine-tuned RoBERTa delivered best accuracy (87.8%).

4.4 Building Neural Network Models

The deep learning and transformer implementations emphasised careful architecture design and training optimisation.

- Sequential Models (LSTM, BiLSTM): Implemented in TensorFlow/Keras with embedding layers to transform tokens into dense vectors. Dropout layers were added for regularisation. Training epochs were limited (5–10) to prevent overfitting.
- Transformers (BERT, RoBERTa): HuggingFace Transformers library used. Fine-tuning was performed using AdamW with learning rate = 2e-5, batch size = 16, over 3 epochs. Frozen ablations were also tested by locking encoder layers and training only the classifier head.
- Regularisation: Dropout layers in both RNNs and transformers reduced overfitting. Learning rate schedulers were applied in transformers for stable convergence.

This staged progression demonstrated the architectural differences between recurrent and transformer networks and their impact on sentiment classification.

4.5 Verification and Validation

- Verification: Ensured correctness of model training and evaluation pipelines through unit testing within the scripts (e.g., validating TF-IDF outputs, sequence lengths, label encoding).
- Validation: Performance monitored on validation subsets using precision, recall, F1-score, and confusion matrices.
- Cross-checking: Results from Python classification reports (classification_report) were matched with confusion matrices to confirm consistency.
- Ablation Verification: Frozen vs fine-tuned comparisons validated the hypothesis that transformer fine-tuning is critical for high performance.

4.6 Results Presentation

Results were presented in the form of:

- Classification Reports: Provided detailed per-class precision, recall, and F1-scores.
- Confusion Matrices: Visualised misclassification patterns, highlighting challenges in negative sentiment detection.
- Comparison Tables: Summarised performance across models, showing transformer superiority.
- Charts: Accuracy comparison chart across all models, highlighting RoBERTa's advantage.
  Overall, the results confirmed that fine-tuned transformers outperformed all other models, with RoBERTa emerging as the best system (87.8% accuracy). However, BiLSTM and Random Forest remained competitive baselines, balancing accuracy and efficiency.

5.0 System Architecture and Evaluation

5.1 System Architecture

The architecture of the project followed a modular workflow, ensuring that each stage of the pipeline could be validated independently before integration:

1. Data Layer – Dataset sourced from Kaggle's *Sentiment Analysis Dataset* containing labelled tweets. Data was stored in CSV format and accessed via pandas. Stratified train-validation-test splits were created to maintain label distribution.
2. Preprocessing Layer – Implemented text cleaning, tokenisation, and feature extraction:
   - TF-IDF vectorisation for classical ML.
   - Word-level embeddings for LSTM and BiLSTM.
   - Subword tokenisation (WordPiece/Byte-Pair Encoding) for BERT and RoBERTa.
3. Modeling Layer – Comprised three categories:
   - Classical ML models (Naïve Bayes, Random Forest).
   - Deep learning models (LSTM, BiLSTM).
   - Transformer models (BERT, RoBERTa, both frozen and fine-tuned).

4. Evaluation Layer – Generated quantitative metrics (accuracy, precision, recall, F1-scores) and qualitative outputs (confusion matrices). These results were compared systematically across models to determine the most effective approach.

5.2 Evaluation Metrics

Evaluation of all models was standardised using the following metrics:

- Accuracy – Proportion of correctly classified tweets.
- Precision – Ability of the model to avoid false positives.
- Recall – Ability to correctly identify each sentiment class.
- F1-score – Harmonic mean of precision and recall, balancing both metrics.
- Confusion Matrices – Visualised distribution of predictions across classes, revealing error patterns.

The use of multiple metrics was critical due to dataset imbalance (negative class under-represented). Accuracy alone would mask these disparities, whereas recall and F1-scores provided deeper insights.

5.3 Comparative Results with Supporting Evidence

5.3.1 Naïve Bayes and Random Forest

The Naïve Bayes classifier (Figure 1: Classification Report; Figure 3: Confusion Matrix) achieved 68% accuracy, with the classification report showing weak precision and recall for the negative class, reflecting its reliance on simple word-frequency assumptions. Random Forest (Figure 2: Classification Report; Figure 4: Confusion Matrix), however, reached 84% accuracy and produced more balanced metrics. Its confusion matrix indicated that neutral and positive classes were recognised reliably, though misclassifications persisted between negative and neutral.

```
=== Tuning Naive Bayes ===
Best parameters: {'alpha': 0.1}
Best macro-F1: 0.6543

Classification Report:
              precision    recall  f1-score   support

           0       0.72      0.52      0.61     11021
           1       0.67      0.63      0.65     16556
           2       0.67      0.81      0.73     20609

    accuracy                           0.68     48186
   macro avg       0.69      0.65      0.66     48186
weighted avg       0.68      0.68      0.68     48186
```

Figure 1: NaïveBayes Classification Report

```
=== Tuning Random Forest ===
Best parameters: {'max_depth': None, 'min_samples_split': 5, 'n_estimators': 200}
Best macro-F1: 0.8031

Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.73      0.78     11021
           1       0.81      0.89      0.85     16556
           2       0.86      0.86      0.86     20609

    accuracy                           0.84     48186
   macro avg       0.84      0.83      0.83     48186
weighted avg       0.84      0.84      0.84     48186

arpan@Arpans-MacBook-Pro ~ %
```

Figure 2: Random Forest Classification Report

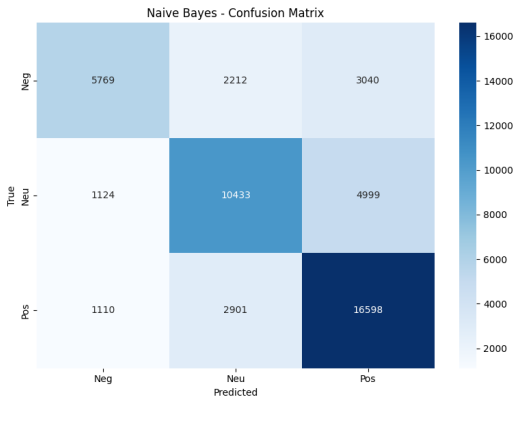

Figure 3: NaïveBayes Confusion Matrix

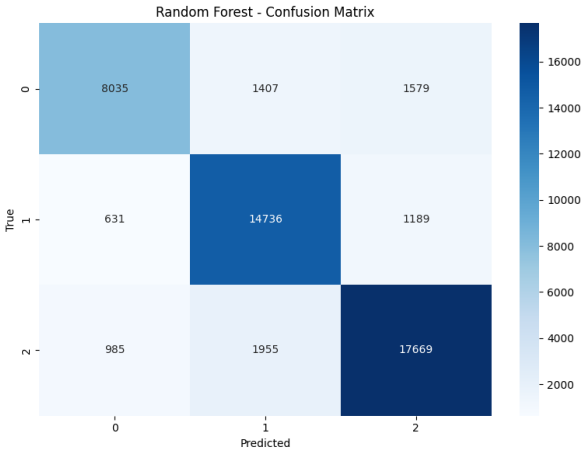

Figure 4: Random Forest Confusion Matrix

5.3.2 LSTM (Long- Short Term Memory)

The LSTM model (Figure 5: Classification Report; Figure 6: Confusion Matrix) achieved only 43% accuracy. The classification report demonstrated a high recall for the positive class but near-zero recall for negative tweets, showing that the model defaulted heavily toward the majority class. The confusion matrix confirmed this collapse, with most predictions falling into the positive category.

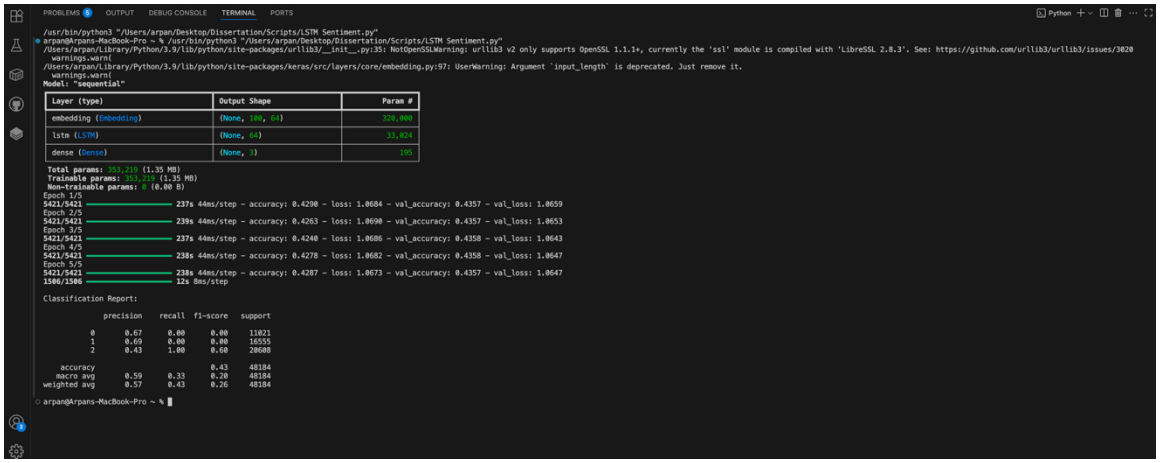

Figure 6: LSTM Classification Report

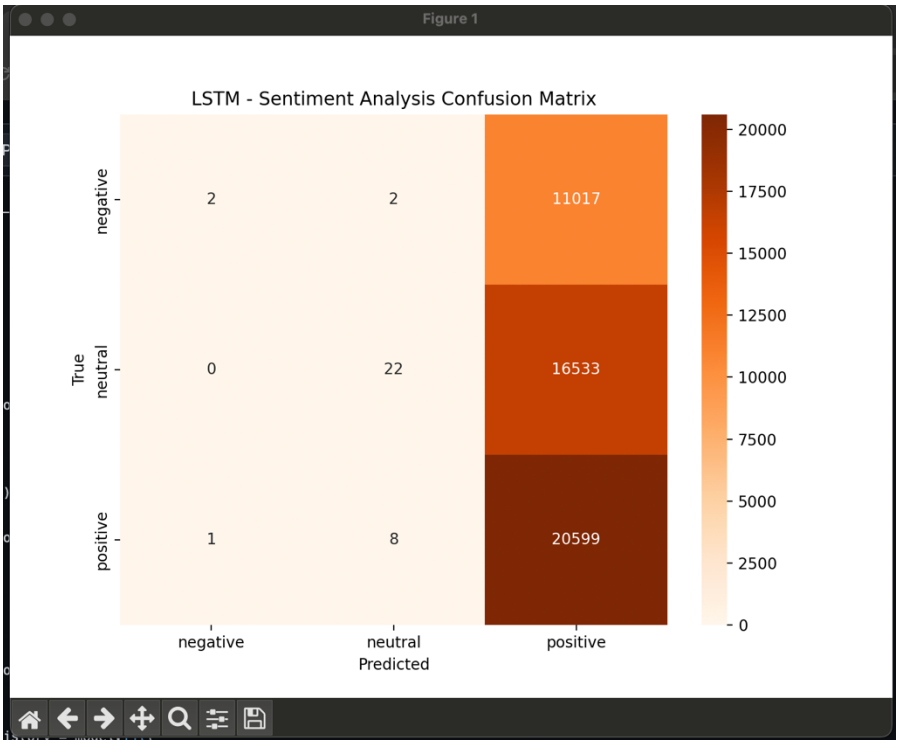

Figure 5: LSTM Confusion Matrix

5.3.3 BiLSTM (Bidirectional Long- Short Term Memory)

The BiLSTM model (Figure 7: *Classification Report*; Figure 8: *Confusion Matrix*) showed significant improvement, achieving 84% accuracy. The classification report displayed balanced precision and recall across all three classes, with F1-scores above 0.78. The confusion matrix demonstrated that although negative and neutral classes were occasionally confused, positive sentiment was classified with high confidence. This highlighted BiLSTM's advantage in capturing bidirectional context, making it a strong non-transformer baseline.

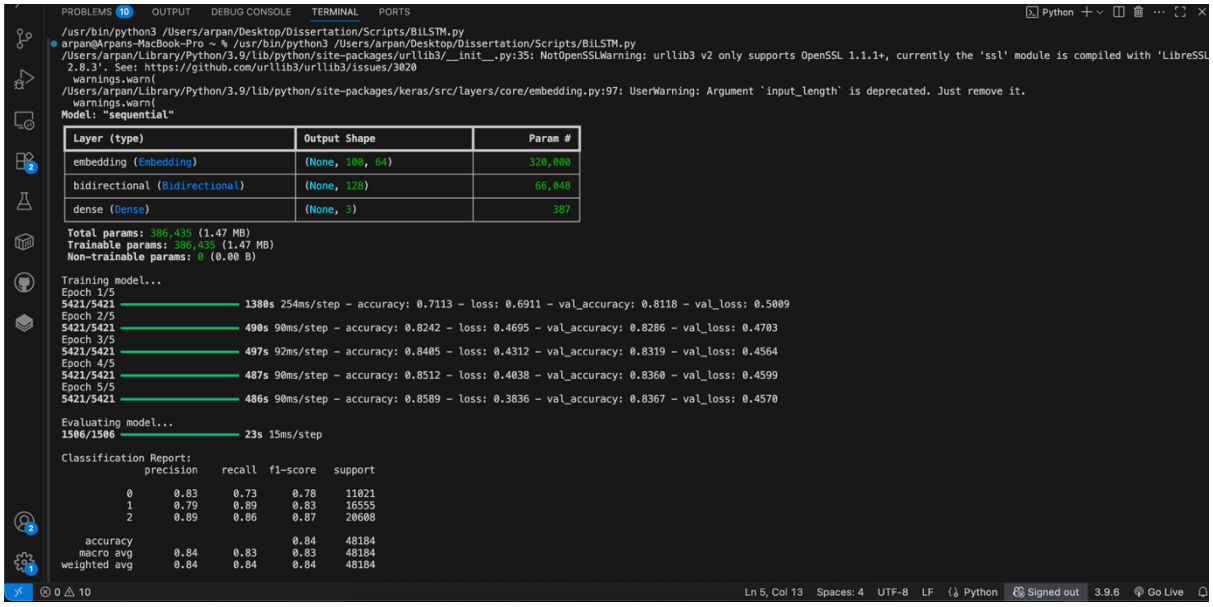

Figure 7: BiLSTM Classification Report

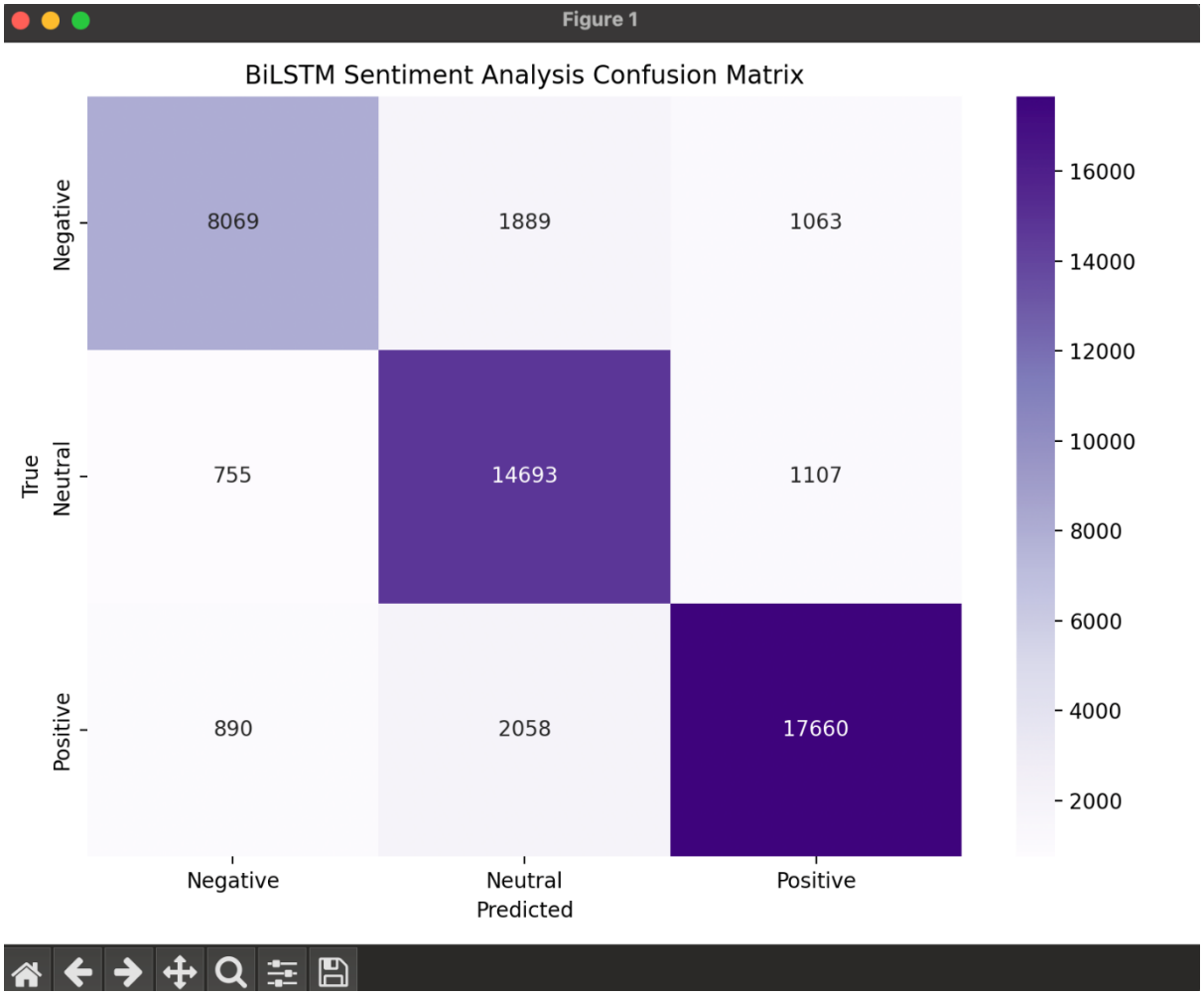

Figure 8: BiLSTM Confusion Matrix

5.3.4 BERT (Bidirectional Encoder Representations from Transformers)

BERT experiments included both frozen and fine-tuned settings as:

- Frozen BERT (Figure 9: Classification Report and Figure 10: Confusion Matrix) collapsed, predicting almost all inputs as positive. This is evident from the confusion matrix where nearly the entire distribution lies in the positive row. Its accuracy of 48.6% reinforced that pre-trained embeddings alone are insufficient for this dataset.

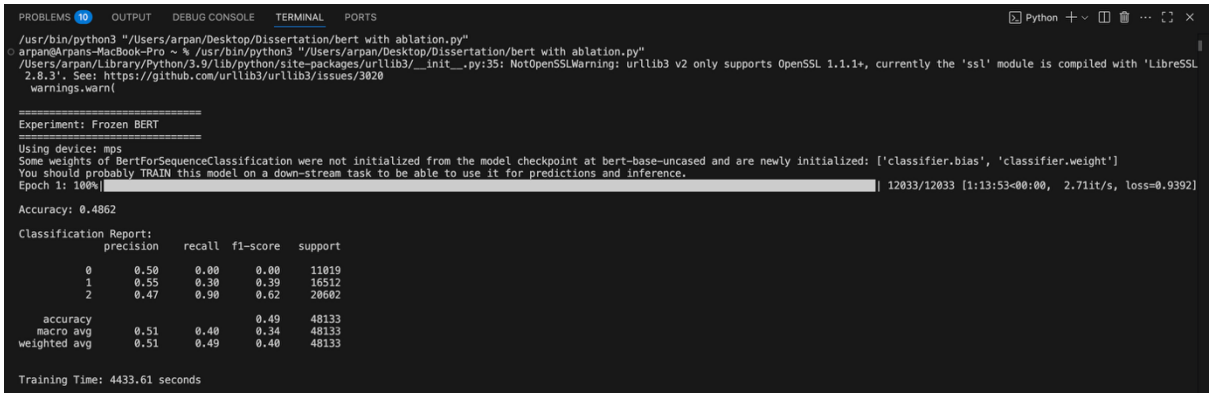

Figure 9: Frozen BERT Classification Report

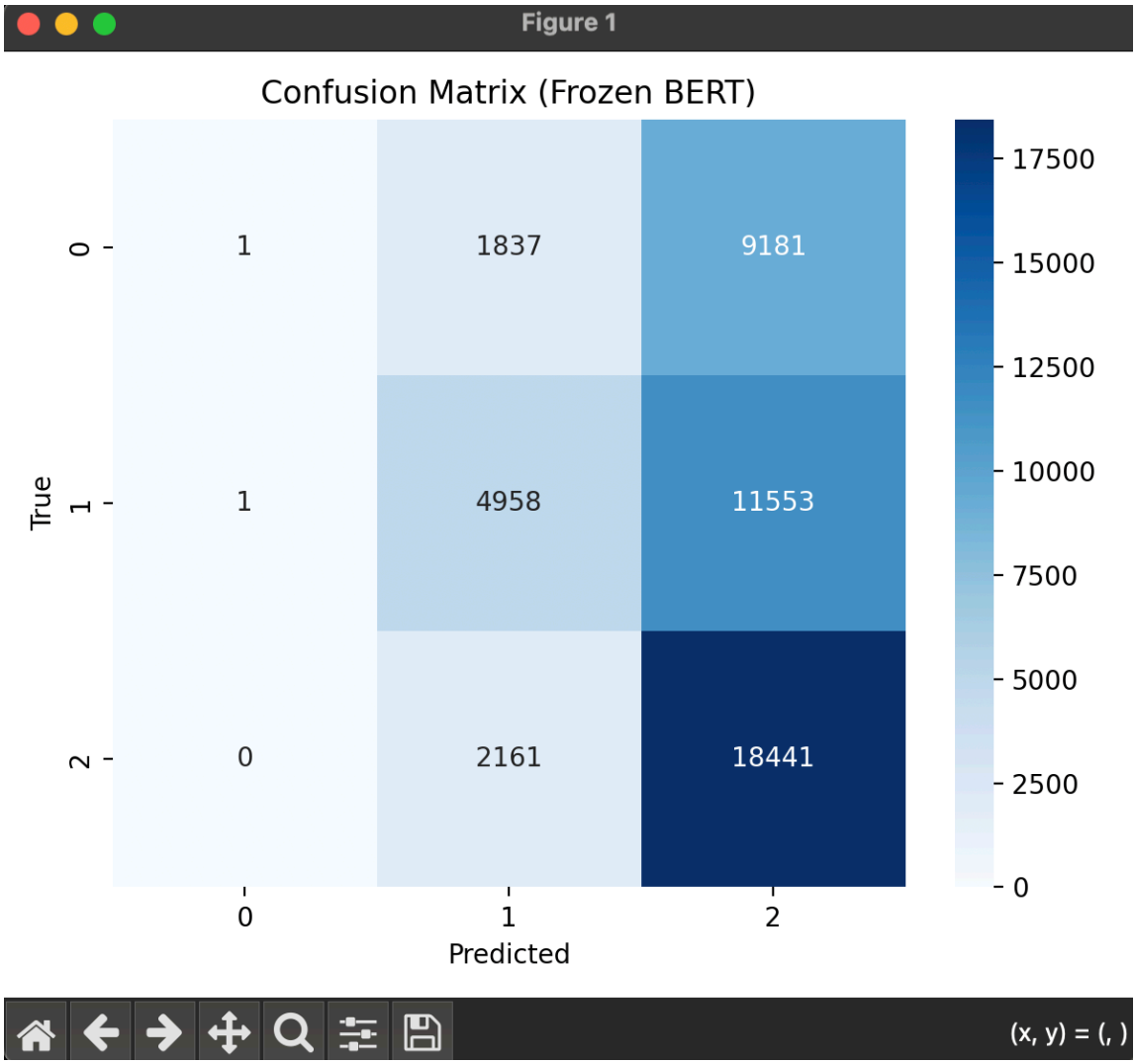

Figure 10: Frozen BERT Confusion Matrix

- Fine-tuned BERT (Figure 11: Classification Report & Figure 12: Confusion Matrix) achieved 85.8% accuracy, with macro F1 ≈ 0.85. The classification report shows balanced performance, while the confusion matrix illustrates reduced confusion between neutral and positive compared to BiLSTM.

```
============================
Experiment: Full Fine-Tuning
============================
Using device: mps
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Epoch 1: 100%|████████████████████████████████████████████████████████| 12033/12033 [3:56:17<00:00,  1.18s/it, loss=0.7203]

Accuracy: 0.8580

Classification Report:
              precision    recall  f1-score   support

           0       0.76      0.87      0.81     11019
           1       0.87      0.83      0.85     16512
           2       0.91      0.87      0.89     20602

    accuracy                           0.86     48133
   macro avg       0.85      0.86      0.85     48133
weighted avg       0.86      0.86      0.86     48133


Training Time: 14177.55 seconds


========= FINAL COMPARISON =========
Frozen BERT - Accuracy: 0.4862, Training Time: 4433.61s
Full Model - Accuracy: 0.8580, Training Time: 14177.55s

Accuracy Improvement from Freezing to Full: 0.3719
Misclassification pattern differences can be further analyzed in the confusion matrices above.
arpan@Arpans-MacBook-Pro ~ %
```

Figure 1 1: Fine- Tuned BERT Classification Report

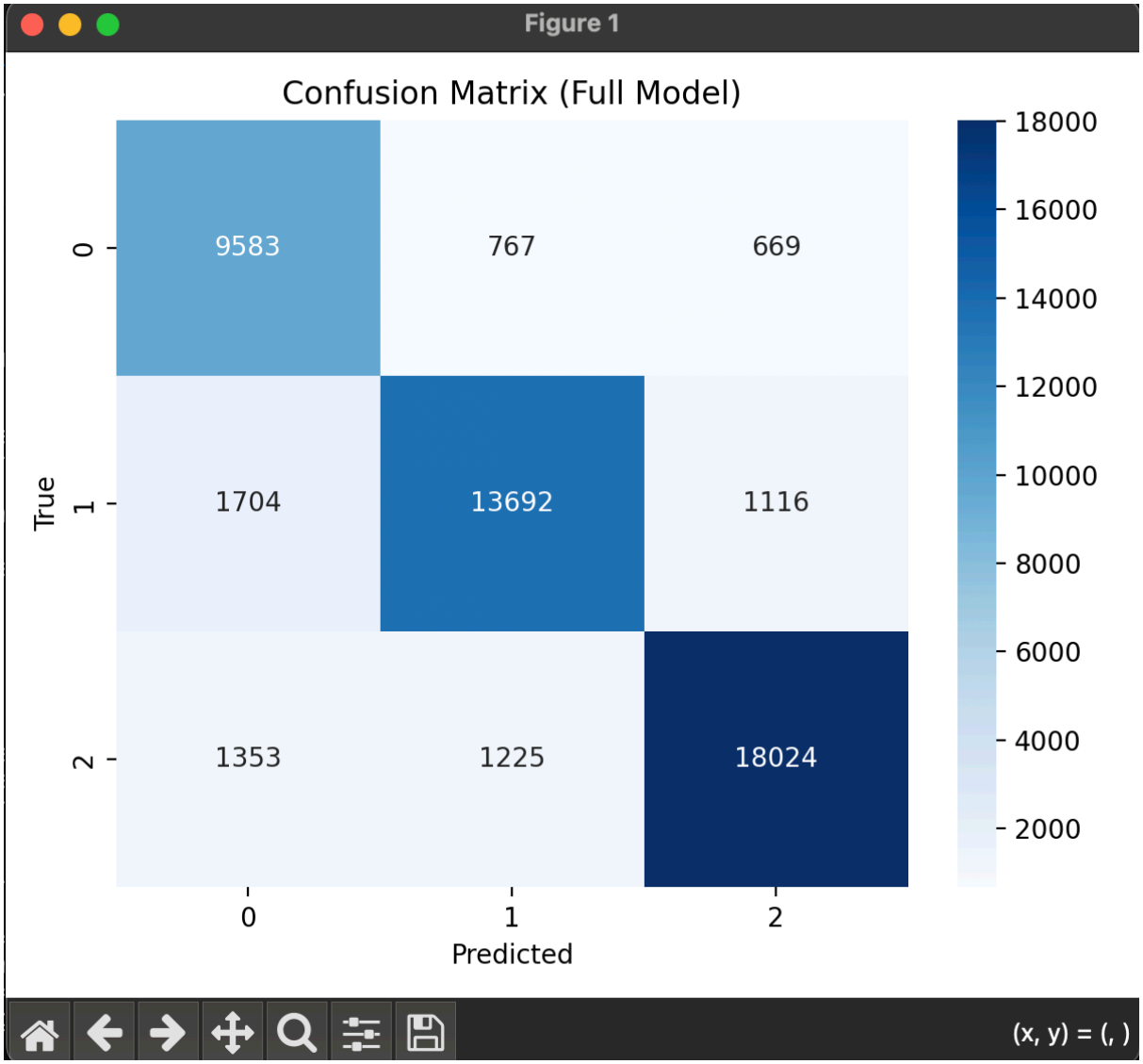

Figure 1 2: Fine- Tuned BERT Confusion Matrix

5.3.5 RoBERTa (Robustly Optimized BERT Pretraining Approach)

RoBERTa experiments included both frozen and fine-tuned as:

- Frozen RoBERTa (Figure 13: Confusion Matrix & Figure 14: Report) achieved 58.1% accuracy, better than frozen BERT but still weak, with the negative class largely misclassified as neutral.

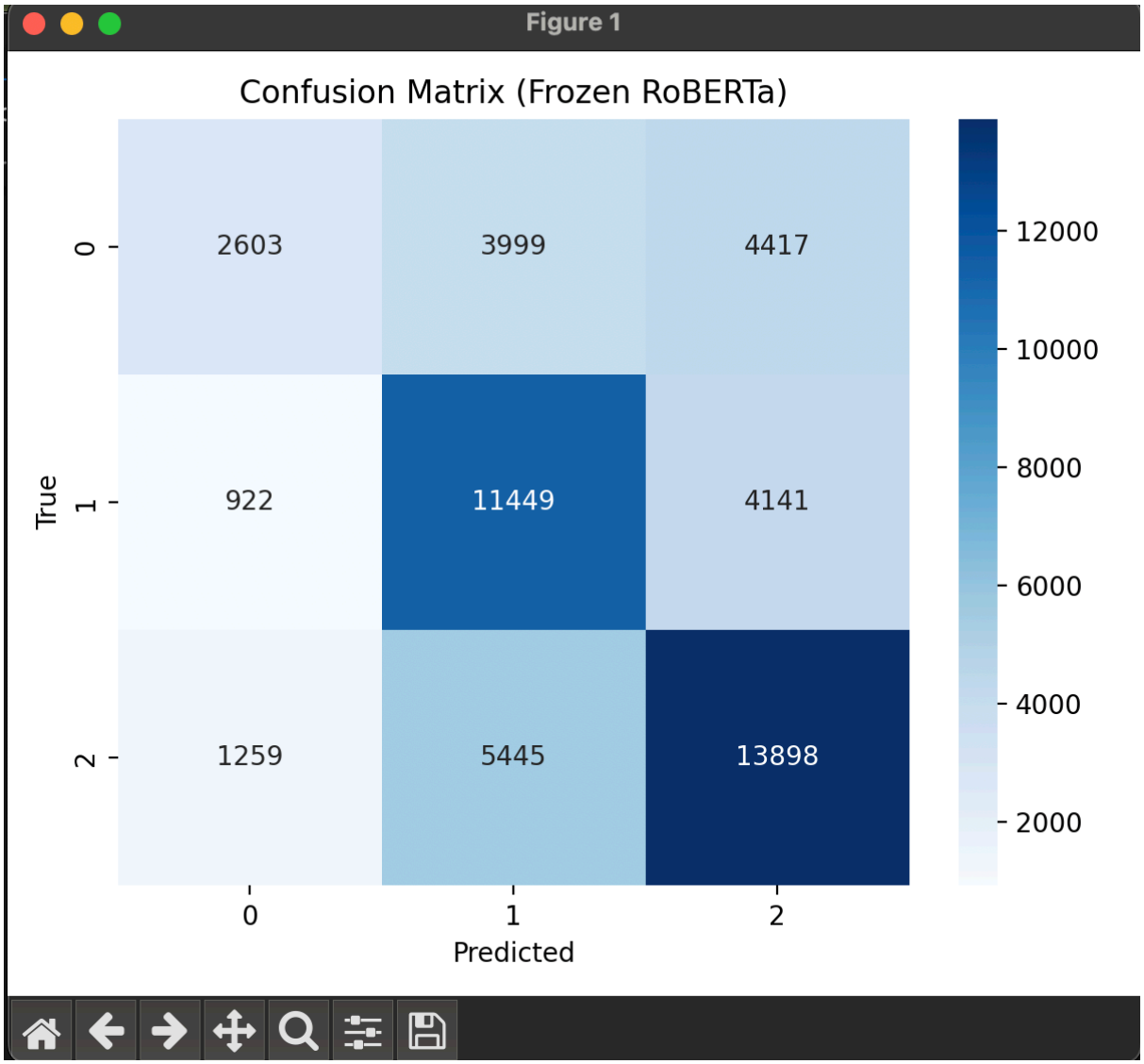

Figure 13: Frozen RoBERTa Confusion Matrix

```
arpan@Arpans-MacBook-Pro ~ % /usr/bin/python3 "/Users/arpan/Desktop/Dissertation/RoBERTa
with Ablation.py"
/Users/arpan/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL
 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(

============================
Experiment: Frozen RoBERTa
============================
Using device: mps
Some weights of RobertaForSequenceClassification were not initialized from the model checkpoint at roberta-base and are newly initialized: ['classifier.dense.bias', 'classifier.dense.weight', 'c
lassifier.out_proj.bias', 'classifier.out_proj.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Epoch 1: 100%|████████████████| 12033/12033 [1:27:47<00:00,  2.28it/s, loss=0.9841]
Epoch 2: 100%|████████████████| 12033/12033 [1:14:26<00:00,  2.69it/s, loss=1.0173]
Epoch 3: 100%|████████████████| 12033/12033 [1:14:57<00:00,  2.68it/s, loss=0.8921]

Accuracy: 0.5807

Classification Report:
              precision    recall  f1-score   support

           0       0.54      0.24      0.33     11019
           1       0.55      0.69      0.61     16512
           2       0.62      0.67      0.65     20602

    accuracy                           0.58     48133
   macro avg       0.57      0.53      0.53     48133
weighted avg       0.58      0.58      0.56     48133

Training Time: 14230.54 seconds
```

Figure 14: Frozen RoBERTa Classification Report

- Fine-tuned RoBERTa (Figure 16: Classification Report & Figure 15: Confusion Matrix) delivered the best results overall at 87.8% accuracy. The classification report shows F1-scores of 0.84 (negative), 0.87 (neutral), and 0.90 (positive), demonstrating a well-balanced model. The confusion matrix confirms minimal overlap between classes, particularly strong in distinguishing neutral from positive — a known challenge in sentiment analysis.

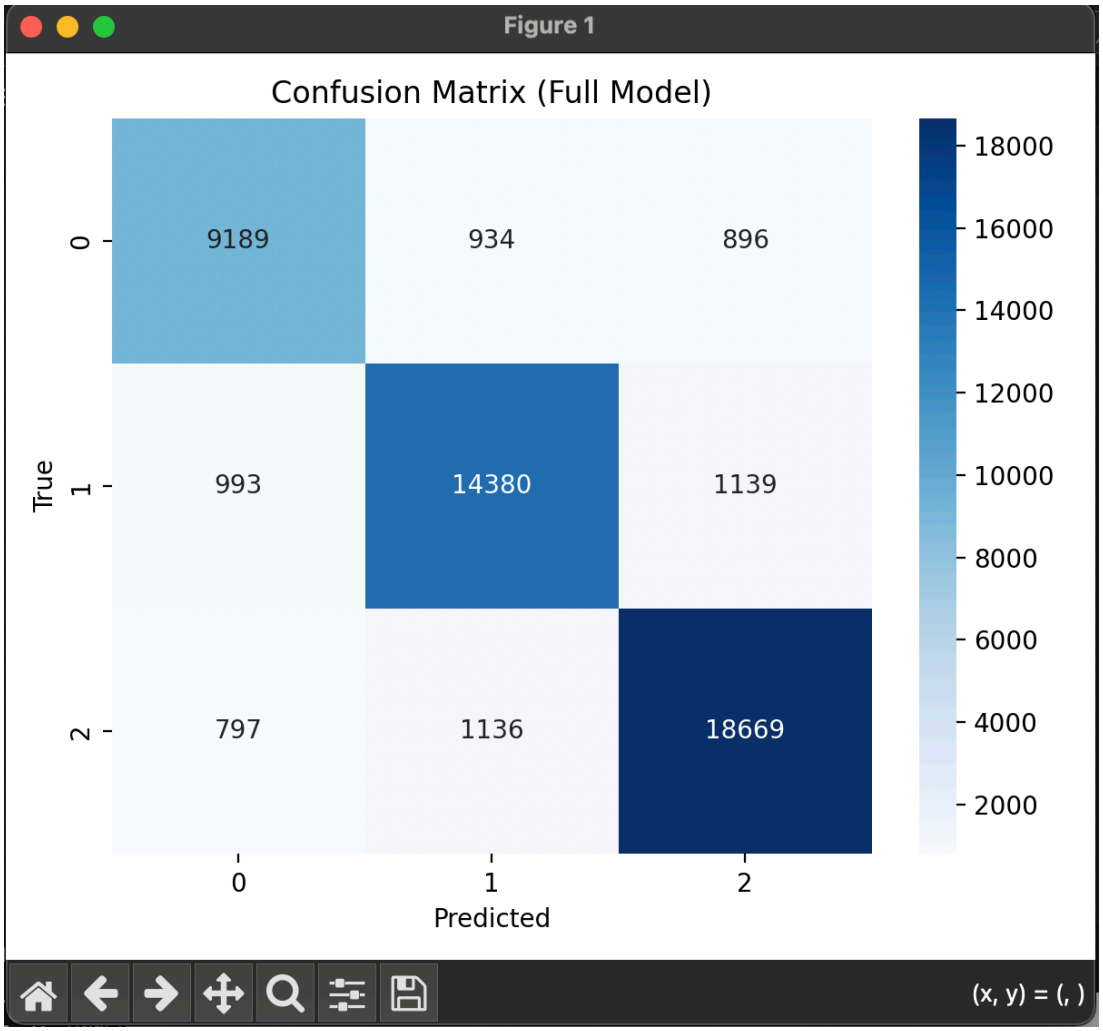

Figure 15: Fine- Tuned RoBERTa Confusion Matrix

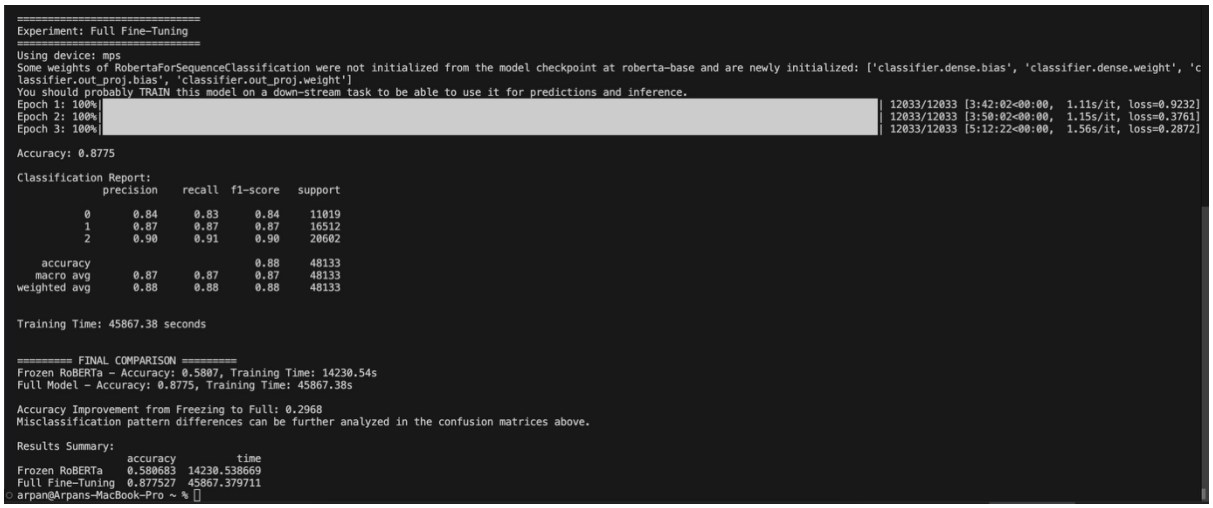

Figure 16: Fine- Tuned RoBERTa Classification Report

5.4 Discussion of Results

- Frozen vs Fine-Tuned: Figures 9, 10, 13 and 14 clearly demonstrate the collapse of frozen transformers. Both frozen BERT and RoBERTa predicted almost exclusively the majority class, highlighting that fine-tuning is not optional but essential.
- Transformer Superiority: Figures 11, 12, 15 and 16 show the robustness of fine-tuned transformers, with RoBERTa outperforming BERT in both accuracy and class-level balance.
- RNN Models: Figures 7 and 8 illustrate BiLSTM's competitive performance, nearly matching fine-tuned BERT, though with slightly higher confusion between negative and neutral classes.
- Classical Models: Among the respective Figures (Figure 1,2,3 and 4) confusion matrices and classification report were simpler, classification reports indicated Random Forest's relative strength compared to Naïve Bayes, validating its inclusion as a meaningful baseline.
- Error Trends: Across all confusion matrices, negative sentiment remained the most difficult to classify, often being confused with neutral. This consistent trend suggests that dataset imbalance was a key factor limiting model performance.

Model	**Accuracy**	**Notes**
Naïve Bayes	68%	Struggled with neutral/negative balance.
Random Forest	84%	Competitive, handled neutral/positive well.
LSTM	43%	Collapsed, biased toward positives.
BiLSTM	84%	Strong sequential learner, balanced results.
Frozen BERT	48.6%	Collapsed, predicted positives disproportionately.
Fine-tuned BERT	85.8%	Strong performance, balanced across classes.
Frozen RoBERTa	58.1%	Better than frozen BERT, but weak overall.
Fine-tuned RoBERTa	**87.8%**	Best-performing model across all metrics.

Table 1: Comparative Result

6.0 Critical appraisal and Lessons Learned

This project gave me the chance to see how sentiment analysis has evolved, from simple classical methods to deep learning and modern transformers. One of the clearest lessons I learned was how essential fine-tuning is for models like BERT and RoBERTa — without it, they were almost unusable, but with it they became the best performers. At the same time, I was surprised by how strong simpler models such as BiLSTM and Random Forest still were, especially when considering their efficiency compared to the heavy cost of transformer training.

There were challenges too. The imbalance in the dataset made negative sentiment hard to classify, and it exposed the limits of some models, particularly LSTM. I also ran into the reality of computational limits, which meant I couldn't test larger models or tune every parameter as much as I would have liked.

Overall, the project taught me to value both performance and practicality, and to think critically about when a complex model is necessary. It also made me more aware of ethical issues, like how bias in data can affect results. More than just testing algorithms, this was a learning journey that has helped me grow as a researcher — teaching me the importance of transparency, reproducibility, and reflection in every stage of the process.

7.0 APPENDICES

7.1 REFERENCES

[1] Esuli, A., & Sebastiani, F. (2006). *SentiWordNet: A publicly available lexical resource for opinion mining*. Proceedings of LREC, 417–422.
[2] Pang, B., Lee, L., & Vaithyanathan, S. (2002). *Thumbs up? Sentiment classification using machine learning techniques*. Proceedings of the ACL-EMNLP, 79–86.
[3] McCallum, A., & Nigam, K. (1998). *A comparison of event models for Naïve Bayes text classification*. AAAI-98 Workshop on Learning for Text Categorization, 41–48.
[4] Breiman, L. (2001). *Random forests*. Machine Learning, 45(1), 5–32.
[5] Hochreiter, S., & Schmidhuber, J. (1997). *Long short-term memory*. Neural Computation, 9(8), 1735–1780.
[6] Graves, A., & Schmidhuber, J. (2005). *Framewise phoneme classification with bidirectional LSTM and other neural network architectures*. Neural Networks, 18(5–6), 602–610.
[7] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of deep bidirectional transformers for language understanding*. Proceedings of NAACL-HLT, 4171–4186.
[8] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., et al. (2019). *RoBERTa: A robustly optimized BERT pretraining approach*. arXiv preprint arXiv:1907.11692.
[9] Imran, A., Mitra, P., & Srivastava, J. (2016). *Cross-cultural polarity and emotion detection on Twitter*. Proceedings of the International Conference on Social Informatics, 13–22.
[10] Sun, C., Qiu, X., Xu, Y., & Huang, X. (2019). *How to fine-tune BERT for text classification?* China National Conference on Chinese Computational Linguistics, 194–206.

7.2 Training Hardware

- Processor: Apple M1, 8-core CPU.
- Graphics Processing Unit (GPU): Integrated 8-core Apple GPU.
- Memory (RAM): 8 GB unified memory.
- Operating System: macOS Monterey (64-bit).

7.3 Software

- Python 3.9.6
- cv2 4.5.4
- Keras 2.9.0
- numpy 1.23.1
- pandas 1.3.4
- Pillow 8.4.0
- scikit-image 0.19.3
- scikit-learn 1.1.2
- tensorflow 2.9.1
- torch 1.12.0
- torchvision 0.13.0

7.4 Github Link

https://github.com/arpanneupane/Dissertation

7.5 Dataset

https://www.kaggle.com/datasets/abdelmalekeladjelet/sentiment-analysis-dataset

sentiment_data.csv

7.6 Codes with Respective Files

7.6.1 Classical Models

```
 # Code for Classical Models (Naive Bayes and Random Forest) with Tuning
import os
import pandas as pd
import numpy as np
import re
import string
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.naive_bayes import MultinomialNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
import joblib
# Download NLTK resources
nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)
# Load dataset
script_dir = os.path.dirname(os.path.abspath(__file__))
csv_path = os.path.join(script_dir, "sentiment_data.csv")
df = pd.read_csv(csv_path)
# Text preprocessing
def preprocess(text):
    if pd.isna(text):
        return ""
    text = str(text).lower()
    text = re.sub(r"\d+", "", text)
    text = text.translate(str.maketrans('', '', string.punctuation))
    tokens = word_tokenize(text)
    stop_words = set(stopwords.words('english'))
    tokens = [word for word in tokens if word not in stop_words]
    return ' '.join(tokens)
df['cleaned'] = df['Comment'].apply(preprocess)
df = df[df['cleaned'].str.strip() != '']
# TF-IDF with unigrams and bigrams
vectorizer = TfidfVectorizer(ngram_range=(1, 2), max_features=20000)
X = vectorizer.fit_transform(df['cleaned'])
y = df['Sentiment']
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
# Define models and hyperparameters for tuning
models = {
    'Naive Bayes': {
        'model': MultinomialNB(),
        'params': {
            'alpha': [0.1, 0.5, 1.0, 2.0]
        }
```

```
    },
    'Random Forest': {
        'model': RandomForestClassifier(class_weight='balanced', random_state=42),
        'params': {
            'n_estimators': [50, 100, 200],
            'max_depth': [None, 10, 20],
            'min_samples_split': [2, 5, 10]
        }
    }
}
# Train and tune each model
for model_name, model_info in models.items():
    print(f"\n=== Tuning {model_name} ===")
    clf = GridSearchCV(model_info['model'], model_info['params'], cv=3, scoring='f1_macro', n_jobs=-1)
    clf.fit(X_train, y_train)

    print(f"Best parameters: {clf.best_params_}")
    print(f"Best macro-F1: {clf.best_score_:.4f}")

    # Evaluate on test set
    y_pred = clf.predict(X_test)
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))

    # Confusion matrix
    labels = sorted(y.unique())
    cm = confusion_matrix(y_test, y_pred, labels=labels)
    plt.figure(figsize=(8,6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=["Neg", "Neu", "Pos"], yticklabels=["Neg", "Neu",
"Pos"])
    plt.xlabel("Predicted")
    plt.ylabel("True")
    plt.title(f"{model_name} - Confusion Matrix")
    plt.tight_layout()
    plt.savefig(f"{model_name.replace(' ', '_')}_confusion_matrix.png")
    plt.show()
```

7.6.2 LSTM Code

```
# Code for LSTM Model
import os
import pandas as pd
import numpy as np
import string
import re
import nltk
import matplotlib.pyplot as plt
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
from tensorflow.keras.preprocessing.text import Tokenizer
```

```
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.utils import to_categorical
import seaborn as sns

# Download NLTK resources (with quiet mode)
nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)

# ========== 1. DATA LOADING WITH ERROR HANDLING ==========
try:
    # Get the directory where the script is located
    script_dir = os.path.dirname(os.path.abspath(__file__))
    csv_path = os.path.join(script_dir, "sentiment_data.csv")

    # Verify file exists before loading
    if not os.path.exists(csv_path):
        raise FileNotFoundError(f"sentiment_data.csv not found at: {csv_path}")

    # Load and clean data
    df = pd.read_csv(csv_path)
    df = df.dropna(subset=['Comment', 'Sentiment'])  # Remove rows with missing values

    # Check if required columns exist
    if not all(col in df.columns for col in ['Comment', 'Sentiment']):
        raise ValueError("CSV must contain both 'Comment' and 'Sentiment' columns")

except Exception as e:
    print(f"Error loading data: {str(e)}")
    exit()

# ========== 2. IMPROVED TEXT PREPROCESSING ==========
def preprocess(text):
    # Handle missing/NaN values
    if pd.isna(text):
        return ""

    # Convert to string in case of numeric input
    text = str(text)

    # Lowercase
    text = text.lower()

    # Remove numbers
    text = re.sub(r"\d+", "", text)

    # Remove punctuation (faster than str.translate)
    text = re.sub(r'[^\w\s]', '', text)

    # Tokenize and remove stopwords
    tokens = word_tokenize(text)
    stop_words = set(stopwords.words('english'))
    tokens = [word for word in tokens if word not in stop_words]

    return ' '.join(tokens)

# Apply preprocessing
```

```
df['cleaned'] = df['Comment'].apply(preprocess)
df = df[df['cleaned'].str.strip() != '']  # Remove empty strings

# ========== 3. TOKENIZATION AND PADDING ==========
MAX_WORDS = 5000
MAX_LEN = 100

tokenizer = Tokenizer(num_words=MAX_WORDS, oov_token="<OOV>")
tokenizer.fit_on_texts(df['cleaned'])
sequences = tokenizer.texts_to_sequences(df['cleaned'])
padded_sequences = pad_sequences(sequences, maxlen=MAX_LEN, padding='post')

# ========== 4. PREPARE LABELS ==========
# Convert string labels to numerical if needed
if df['Sentiment'].dtype == 'object':
    sentiment_map = {'negative': 0, 'neutral': 1, 'positive': 2}
    df['Sentiment'] = df['Sentiment'].map(sentiment_map)

labels = to_categorical(df['Sentiment'])

# ========== 5. TRAIN-TEST SPLIT ==========
X_train, X_test, y_train, y_test = train_test_split(
    padded_sequences,
    labels,
    test_size=0.2,
    random_state=42,
    stratify=df['Sentiment']  # Maintain class distribution
)

# ========== 6. LSTM MODEL ==========
model = Sequential([
    Embedding(input_dim=MAX_WORDS, output_dim=64, input_length=MAX_LEN),
    LSTM(64, dropout=0.2, recurrent_dropout=0.2),  # Added dropout for regularization
    Dense(3, activation='softmax')
])

model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)
model.build(input_shape=(None, MAX_LEN))  # <<< This builds the model
model.summary()

# ========== 7. TRAINING ==========
history = model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)

# ========== 8. EVALUATION ==========
predictions = model.predict(X_test)
y_pred = np.argmax(predictions, axis=1)
y_true = np.argmax(y_test, axis=1)
```

```
print("\nClassification Report:\n")
print(classification_report(y_true, y_pred))

# Confusion matrix with auto-detected labels
label_names = ['negative', 'neutral', 'positive'] if 'Sentiment' in df.columns else ['0', '1', '2']
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Oranges',
    xticklabels=label_names,
    yticklabels=label_names
)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("LSTM - Sentiment Analysis Confusion Matrix")
plt.show()
```

7.6.3 BiLSTM Code

```
import os
import pandas as pd
import numpy as np
import re
import nltk
import matplotlib.pyplot as plt
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM, Dense
from tensorflow.keras.utils import to_categorical
import seaborn as sns

# Download NLTK resources (silent mode)
nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)

# ========== 1. DATA LOADING WITH ERROR HANDLING ==========
try:
    # Get the absolute path to the CSV file
    script_dir = os.path.dirname(os.path.abspath(__file__))
    csv_path = os.path.join(script_dir, "sentiment_data.csv")

    # Verify file exists
    if not os.path.exists(csv_path):
        raise FileNotFoundError(f"File not found: {csv_path}\n"
                                f"Please ensure 'sentiment_data.csv' is in the same directory as your script.")
```

```
    # Load and clean data
    df = pd.read_csv(csv_path)
    df = df.dropna(subset=['Comment', 'Sentiment'])  # Remove rows with missing values

    # Check required columns
    if not all(col in df.columns for col in ['Comment', 'Sentiment']):
        raise ValueError("CSV must contain both 'Comment' and 'Sentiment' columns")

except Exception as e:
    print(f"Error loading data: {str(e)}")
    exit()

# ========== 2. IMPROVED TEXT PREPROCESSING ==========
stop_words = set(stopwords.words('english'))  # Preload for efficiency

def preprocess(text):
    # Handle missing values
    if pd.isna(text):
        return ""

    text = str(text).lower()  # Ensure string and lowercase
    text = re.sub(r"\d+", "", text)  # Remove numbers
    text = re.sub(r'[^\w\s]', '', text)  # Remove punctuation (faster than translate)
    tokens = word_tokenize(text)
    tokens = [word for word in tokens if word not in stop_words]
    return ' '.join(tokens)

df['cleaned'] = df['Comment'].apply(preprocess)
df = df[df['cleaned'].str.strip() != '']  # Remove empty comments

# ========== 3. TOKENIZATION AND PADDING ==========
MAX_WORDS = 5000
MAX_LEN = 100

tokenizer = Tokenizer(num_words=MAX_WORDS, oov_token="<OOV>")
tokenizer.fit_on_texts(df['cleaned'])
sequences = tokenizer.texts_to_sequences(df['cleaned'])
padded_sequences = pad_sequences(sequences, maxlen=MAX_LEN, padding='post')

# ========== 4. LABEL PROCESSING ==========
# Convert string labels to numerical if needed
if df['Sentiment'].dtype == 'object':
    label_map = {label: idx for idx, label in enumerate(df['Sentiment'].unique())}
    df['Sentiment'] = df['Sentiment'].map(label_map)

labels = to_categorical(df['Sentiment'])

# ========== 5. TRAIN-TEST SPLIT ==========
X_train, X_test, y_train, y_test = train_test_split(
    padded_sequences,
    labels,
    test_size=0.2,
    random_state=42,
    stratify=df['Sentiment']  # Maintain class distribution
)

# ========== 6. BiLSTM MODEL ==========
```

```
model = Sequential([
    Embedding(input_dim=MAX_WORDS, output_dim=64, input_length=MAX_LEN),
    Bidirectional(LSTM(64, dropout=0.2, recurrent_dropout=0.2)),  # Added regularization
    Dense(3, activation='softmax')
])

model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

model.build(input_shape=(None, MAX_LEN))  # <<< This builds the model
model.summary()

# ========== 7. TRAINING ==========
print("\nTraining model...")
history = model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)

# ========== 8. EVALUATION ==========
print("\nEvaluating model...")
predictions = model.predict(X_test)
y_pred = np.argmax(predictions, axis=1)
y_true = np.argmax(y_test, axis=1)

print("\nClassification Report:")
print(classification_report(y_true, y_pred))

# Confusion matrix with dynamic labels
label_names = list(label_map.keys()) if 'label_map' in locals() else ["Negative", "Neutral", "Positive"]
plt.figure(figsize=(8, 6))
sns.heatmap(
    confusion_matrix(y_true, y_pred),
    annot=True,
    fmt='d',
    cmap='Purples',
    xticklabels=label_names,
    yticklabels=label_names
)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("BiLSTM Sentiment Analysis Confusion Matrix")
plt.tight_layout()
plt.show()
```

7.6.4 BERT Code

```
import os
import pandas as pd
import numpy as np
import re
import time
import torch
from torch.utils.data import Dataset, DataLoader
from transformers import BertTokenizer, BertForSequenceClassification
from torch.optim import AdamW
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import seaborn as sns
import matplotlib.pyplot as plt
from tqdm.auto import tqdm
from torch.amp import autocast
import warnings
warnings.filterwarnings("ignore")

# ========== 1. DATA LOADING WITH ERROR HANDLING ==========
try:
    script_dir = os.path.dirname(os.path.abspath(__file__))
    csv_path = os.path.join(script_dir, "sentiment_data.csv")

    if not os.path.exists(csv_path):
        raise FileNotFoundError(f"File not found: {csv_path}\n"
                                f"Ensure 'sentiment_data.csv' is in the same directory.")

    df = pd.read_csv(csv_path)
    df = df.dropna(subset=['Comment', 'Sentiment'])

    if not all(col in df.columns for col in ['Comment', 'Sentiment']):
        raise ValueError("CSV must contain both 'Comment' and 'Sentiment' columns")

except Exception as e:
    print(f"Error loading data: {str(e)}")
    exit()

# ========== 2. TEXT PREPROCESSING ==========
def preprocess(text):
    text = str(text).lower()
    text = re.sub(r"http\S+|www\S+", "", text)
    text = re.sub(r"[^a-zA-Z\s]", "", text)
    return text.strip()

df['cleaned'] = df['Comment'].apply(preprocess)
df = df[df['cleaned'] != '']

# ========== 3. DATA PREPARATION ==========
if df['Sentiment'].dtype == 'object':
    label_map = {label: idx for idx, label in enumerate(sorted(df['Sentiment'].unique()))}
    df['Sentiment'] = df['Sentiment'].map(label_map)

train_texts, test_texts, train_labels, test_labels = train_test_split(

    df['cleaned'].tolist(),
    df['Sentiment'].tolist(),
    test_size=0.2,
```

```
    random_state=42,
    stratify=df['Sentiment']

)

# ========== 4. TOKENIZATION AND DATASET ==========
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

class SentimentDataset(Dataset):
    def __init__(self, texts, labels):
        self.encodings = tokenizer(texts, truncation=True, padding=True, max_length=128)
        self.labels = labels

    def __getitem__(self, idx):
        return {
            'input_ids': torch.tensor(self.encodings['input_ids'][idx]),
            'attention_mask': torch.tensor(self.encodings['attention_mask'][idx]),
            'labels': torch.tensor(self.labels[idx])
        }

    def __len__(self):
        return len(self.labels)

# ========== 5. EXPERIMENT FUNCTION ==========
def run_experiment(freeze_bert: bool):
    print(f"\n{'='*30}\nExperiment: {'Frozen BERT' if freeze_bert else 'Full Fine-Tuning'}\n{'='*30}")

    device = torch.device("mps" if torch.backends.mps.is_available() else "cuda" if torch.cuda.is_available() else "cpu")
    print(f"Using device: {device}")

    # Get the number of unique labels from the training labels
    num_labels = len(set(train_labels))  # This replaces the need for label_map

    # Load model
    model = BertForSequenceClassification.from_pretrained(
        'bert-base-uncased',
        num_labels=num_labels  # Use the calculated number of labels
    )

    if freeze_bert:
        for param in model.bert.parameters():
            param.requires_grad = False

    model.to(device)

    train_dataset = SentimentDataset(train_texts, train_labels)
    test_dataset = SentimentDataset(test_texts, test_labels)

    train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
    test_loader = DataLoader(test_dataset, batch_size=16)

    optimizer = AdamW(filter(lambda p: p.requires_grad, model.parameters()), lr=2e-5)

    # Training
    start_time = time.time()
    model.train()
    for epoch in range(3):
        progress_bar = tqdm(train_loader, desc=f"Epoch {epoch+1}")
```

```
        for batch in progress_bar:
            optimizer.zero_grad()
            inputs = {k: v.to(device) for k, v in batch.items()}
            with autocast(device_type='mps', dtype=torch.float16):
                outputs = model(**inputs)
                loss = outputs.loss
            loss.backward()
            optimizer.step()
            progress_bar.set_postfix({'loss': f"{loss.item():.4f}"})
    end_time = time.time()

    # Evaluation
    model.eval()
    preds, true_labels = [], []
    with torch.no_grad():
        for batch in test_loader:
            inputs = {k: v.to(device) for k, v in batch.items() if k != 'labels'}
            outputs = model(**inputs)
            preds.extend(torch.argmax(outputs.logits, axis=1).cpu().numpy())
            true_labels.extend(batch['labels'].cpu().numpy())

    acc = accuracy_score(true_labels, preds)
    print(f"\nAccuracy: {acc:.4f}")

    # For classification report, we need to get the label names
    # Since we don't have label_map, we'll just use numerical labels
    print("\nClassification Report:")
    print(classification_report(true_labels, preds))

    # Confusion matrix with numerical labels
    plt.figure(figsize=(6, 5))
    sns.heatmap(
        confusion_matrix(true_labels, preds),
        annot=True,
        fmt='d',
        cmap='Blues'
    )
    plt.xlabel("Predicted")
    plt.ylabel("True")
    plt.title(f"Confusion Matrix ({'Frozen BERT' if freeze_bert else 'Full Model'})")
    plt.tight_layout()
    plt.show()

    print(f"\nTraining Time: {end_time - start_time:.2f} seconds")
    return acc, end_time - start_time, classification_report(true_labels, preds, output_dict=True)

# ========== 6. RUN BOTH EXPERIMENTS ==========
results = {}

# Run frozen BERT experiment

acc_frozen, time_frozen, report_frozen = run_experiment(freeze_bert=True)  # Removed label_map argument
results['Frozen BERT'] = {'accuracy': acc_frozen, 'time': time_frozen, 'report': report_frozen}

# Run full fine-tuning experiment
acc_full, time_full, report_full = run_experiment(freeze_bert=False)  # Removed label_map argument
results['Full Fine-Tuning'] = {'accuracy': acc_full, 'time': time_full, 'report': report_full}
```

```
acc_frozen, time_frozen, report_frozen = run_experiment(freeze_bert=True, label_map=label_map)

results['Frozen BERT'] = {'accuracy': acc_frozen, 'time': time_frozen, 'report': report_frozen}

# Run full fine-tuning experiment
acc_full, time_full, report_full = run_experiment(freeze_bert=False, label_map=label_map)

results['Full Fine-Tuning'] = {'accuracy': acc_full, 'time': time_full, 'report': report_full}

# ========== 7. COMPARE RESULTS ==========
print("\n\n========== FINAL COMPARISON ==========")
print(f"Frozen BERT - Accuracy: {acc_frozen:.4f}, Training Time: {time_frozen:.2f}s")
print(f"Full Model - Accuracy: {acc_full:.4f}, Training Time: {time_full:.2f}s")

improvement = acc_full - acc_frozen
print(f"\nAccuracy Improvement from Freezing to Full: {improvement:.4f}")
print("Misclassification pattern differences can be further analyzed in the confusion matrices above.")
```

7.6.5 RoBERTa Code

```
import os
import pandas as pd
import numpy as np
import re
import time
import torch
from torch.utils.data import Dataset, DataLoader
from transformers import RobertaTokenizer, RobertaForSequenceClassification
from torch.optim import AdamW
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import seaborn as sns
import matplotlib.pyplot as plt
from tqdm.auto import tqdm
from torch.amp import autocast
import warnings
warnings.filterwarnings("ignore")

# ========== 1. DATA LOADING WITH ERROR HANDLING ==========
try:
    script_dir = os.path.dirname(os.path.abspath(__file__))
    csv_path = os.path.join(script_dir, "sentiment_data.csv")

    if not os.path.exists(csv_path):
        raise FileNotFoundError(f"File not found: {csv_path}\n"
                                f"Ensure 'sentiment_data.csv' is in the same directory.")

    df = pd.read_csv(csv_path)
    df = df.dropna(subset=['Comment', 'Sentiment'])

    if not all(col in df.columns for col in ['Comment', 'Sentiment']):
        raise ValueError("CSV must contain both 'Comment' and 'Sentiment' columns")
```

```
except Exception as e:
    print(f"Error loading data: {str(e)}")
    exit()

# ========== 2. TEXT PREPROCESSING ==========
def preprocess(text):
    text = str(text).lower()
    text = re.sub(r"http\S+|www\S+", "", text)
    text = re.sub(r"[^a-zA-Z\s]", "", text)
    return text.strip()

df['cleaned'] = df['Comment'].apply(preprocess)
df = df[df['cleaned'] != '']

# ========== 3. DATA PREPARATION ==========
if df['Sentiment'].dtype == 'object':
    label_map = {label: idx for idx, label in enumerate(sorted(df['Sentiment'].unique()))}
    df['Sentiment'] = df['Sentiment'].map(label_map)

train_texts, test_texts, train_labels, test_labels = train_test_split(
    df['cleaned'].tolist(),
    df['Sentiment'].tolist(),
    test_size=0.2,
    random_state=42,
    stratify=df['Sentiment']
)

# ========== 4. TOKENIZATION AND DATASET ==========
tokenizer = RobertaTokenizer.from_pretrained('roberta-base')

class SentimentDataset(Dataset):
    def __init__(self, texts, labels):
        self.encodings = tokenizer(texts, truncation=True, padding=True, max_length=128)
        self.labels = labels

    def __getitem__(self, idx):
        return {
            'input_ids': torch.tensor(self.encodings['input_ids'][idx]),
            'attention_mask': torch.tensor(self.encodings['attention_mask'][idx]),
            'labels': torch.tensor(self.labels[idx])
        }

    def __len__(self):
        return len(self.labels)

# ========== 5. EXPERIMENT FUNCTION ==========
def run_experiment(freeze_roberta: bool):
    print(f"\n{'='*30}\nExperiment: {'Frozen RoBERTa' if freeze_roberta else 'Full Fine-Tuning'}\n{'='*30}")

    device = torch.device("mps" if torch.backends.mps.is_available() else "cuda" if torch.cuda.is_available() else "cpu")
    print(f"Using device: {device}")

    # Get the number of unique labels from the training labels
    num_labels = len(set(train_labels))

    # Load RoBERTa model
    model = RobertaForSequenceClassification.from_pretrained(
```

```
    'roberta-base',
    num_labels=num_labels
)

if freeze_roberta:
    # Freeze all RoBERTa parameters except the classifier
    for param in model.roberta.parameters():
        param.requires_grad = False

model.to(device)

train_dataset = SentimentDataset(train_texts, train_labels)
test_dataset = SentimentDataset(test_texts, test_labels)

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16)

optimizer = AdamW(filter(lambda p: p.requires_grad, model.parameters()), lr=2e-5)

# Training
start_time = time.time()
model.train()
for epoch in range(3):  # RoBERTa typically benefits from more epochs than BERT
    progress_bar = tqdm(train_loader, desc=f"Epoch {epoch+1}")
    for batch in progress_bar:
        optimizer.zero_grad()
        inputs = {k: v.to(device) for k, v in batch.items()}
        with autocast(device_type='mps' if device.type == 'mps' else 'cuda', dtype=torch.float16):
            outputs = model(**inputs)
            loss = outputs.loss
        loss.backward()
        optimizer.step()
        progress_bar.set_postfix({'loss': f"{loss.item():.4f}"})
end_time = time.time()

# Evaluation
model.eval()
preds, true_labels = [], []
with torch.no_grad():
    for batch in test_loader:
        inputs = {k: v.to(device) for k, v in batch.items() if k != 'labels'}
        outputs = model(**inputs)
        preds.extend(torch.argmax(outputs.logits, axis=1).cpu().numpy())
        true_labels.extend(batch['labels'].cpu().numpy())

acc = accuracy_score(true_labels, preds)
print(f"\nAccuracy: {acc:.4f}")

print("\nClassification Report:")
print(classification_report(true_labels, preds))

# Confusion matrix with numerical labels
plt.figure(figsize=(6, 5))
sns.heatmap(
    confusion_matrix(true_labels, preds),
    annot=True,
    fmt='d',
    cmap='Blues'
```

```
    )
    plt.xlabel("Predicted")
    plt.ylabel("True")
    plt.title(f"Confusion Matrix ({'Frozen RoBERTa' if freeze_roberta else 'Full Model'})")
    plt.tight_layout()
    plt.show()

    print(f"\nTraining Time: {end_time - start_time:.2f} seconds")
    return acc, end_time - start_time, classification_report(true_labels, preds, output_dict=True)

# ========== 6. RUN BOTH EXPERIMENTS ==========
results = {}

# Run frozen RoBERTa experiment
acc_frozen, time_frozen, report_frozen = run_experiment(freeze_roberta=True)
results['Frozen RoBERTa'] = {'accuracy': acc_frozen, 'time': time_frozen, 'report': report_frozen}

# Run full fine-tuning experiment
acc_full, time_full, report_full = run_experiment(freeze_roberta=False)
results['Full Fine-Tuning'] = {'accuracy': acc_full, 'time': time_full, 'report': report_full}

# ========== 7. COMPARE RESULTS ==========
print("\n\n========== FINAL COMPARISON ==========")
print(f"Frozen RoBERTa - Accuracy: {acc_frozen:.4f}, Training Time: {time_frozen:.2f}s")
print(f"Full Model - Accuracy: {acc_full:.4f}, Training Time: {time_full:.2f}s")

improvement = acc_full - acc_frozen
print(f"\nAccuracy Improvement from Freezing to Full: {improvement:.4f}")
print("Misclassification pattern differences can be further analyzed in the confusion matrices above.")

# Save results for further analysis
results_df = pd.DataFrame.from_dict(results, orient='index')
print("\nResults Summary:")
print(results_df[['accuracy', 'time']])
```